Session 9 practical work

On completion of these activities, you should:

- Understand the concept of hash functions and hash values.
- Become familiar with the input and output of hashing algorithms like MD5.
- Understand the importance of strong passwords.
- Explore passwords and discover if they are secure or not.
- Understand the concept of salt when generating hash values.

Hash functions

Hash functions are functions or algorithms that are used to map or convert data (think of strings or text) from an arbitrary size into fixed length values called **hash values**. The values should have several properties. Two of the most important properties ones are:

(1) It is not easy to find the original data from the hash value, so they can be considered as one-way functions from the data to the hash value,
(2) It should be very difficult to generate two identical hash values from different original data.

For the first property, think of the *modulo operation* that returns the remainder of a division. So "`5 mod 2`" returns `1` as `2x2+1 = 5` and "`9 mod 3`" would return `0` as `3x3 =9`. If we only know the output of the operation, say 2, we cannot know which were the two original numbers, for example `5 mod 2 = 1`, `9 mod 2 = 1` or `1 mod 2 = 1`. Therefore, if you know the answer, you cannot go back and determine the two values that created the remainder. This example illustrates also the second property. Those three cases produced the same output. This is known as a collision, and ideally, it should be very difficult to generate two identical hash values from different input data. When generating hash values, you want these to be long, as the longer it is, the more combinations that are possible and the harder it is to crack.

The MD5 (Message Digest algorithm version 5) is a widely known algorithm used to produce hash values from strings of any length. The hash values produced with MD5 are 128 bits long and are normally shown not as binary values but as hexadecimal, which are shorter and easier to read. An example of an MD5 hash value is `0cc175b9c0f1b6a831c399e269772661`, this hash value corresponds to the string "*a*". Another example is `7fc56270e7a70fa81a5935b72eacbe29` which corresponds to "*A*". Yet another hash value is `301aa9f5ba9089a85a94a30b83da50ed` which corresponds to "*To be, or not to be, that is the question: Whether 'tis nobler in the mind to suffer The slings and arrows of outrageous fortune, Or to take Arms against a Sea of troubles, And by opposing end them: to die, to sleep; No more; and by a sleep, to say we end The heart-ache, and the thousand natural shocks That Flesh is heir to? 'Tis a consummation*". Notice, the hash is always the same length irrespective of the input data.

MD5 was designed by Ronald Rivest in 1991 to replace an earlier hash function MD4 (and there was an earlier MD2) and was designed to be used as a cryptographic hash function.

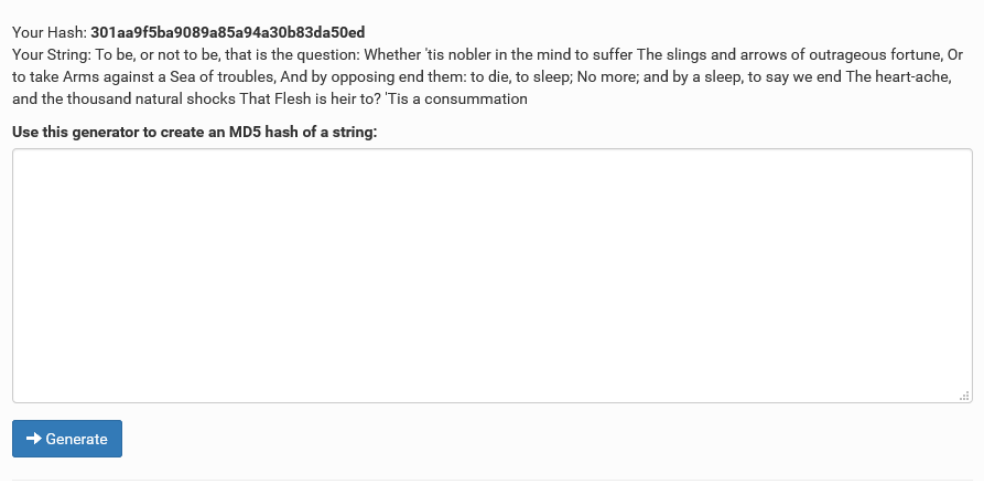
However, as with many other functions, it was found to suffer from vulnerabilities, and it has been replaced by other functions.

Task 1: Try the MD5 hash algorithm.

There are many websites where you can use the MD5 algorithm to hash a given string, one of them is:

`https://www.md5hashgenerator.com/`

a) Go to the website and experiment with a few passwords. Try some short and some long ones.

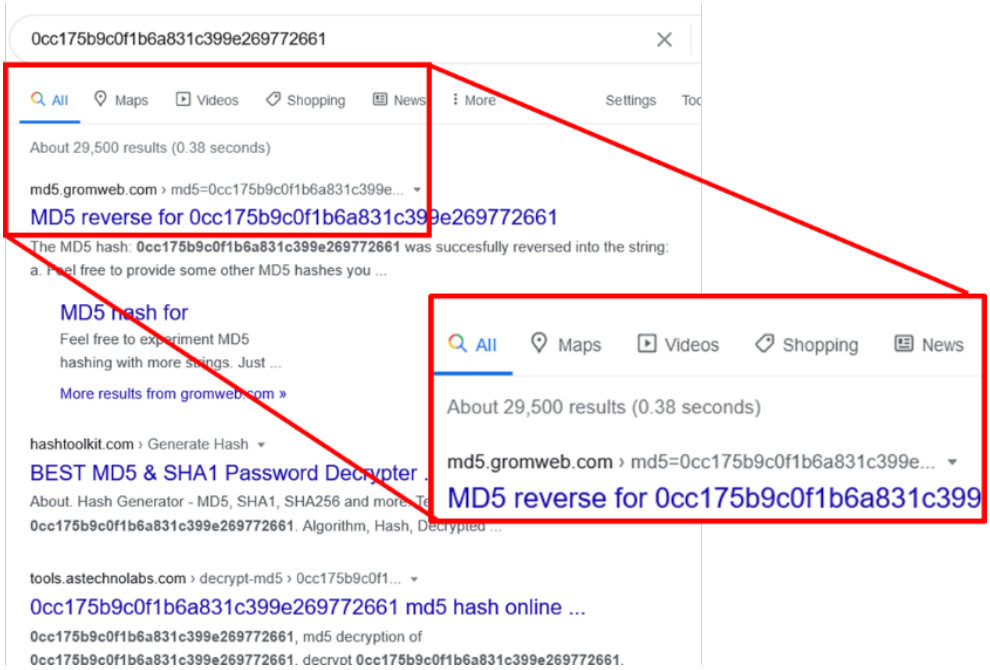


b) Experiment with strings that are similar like `password1` and `password2`.
c) Can you find two different input strings that can generate the same MD5 hash value?

Task 2: Never trust a simple password …

So, what is the problem with using `password` as your password? Passwords should never be stored as plain text, they are hashed and stored as a hash value, so that when someone logs in to a computer or site, they type their password, which is hashed and the hash value is compared with the stored hash value, generated at the time of registration. It may happen that the stored hashed passwords may be stolen by hackers. Since the hash is a one-way function, it is not possible to find out the password from the hash value. However, if the password were not very strong (and by that take all the words in a dictionary and many many more), hackers could quickly find out which password generated the corresponding hash value. How? Well, you do not need to programme something sophisticated, just google it!

a) Generate a hash value with a simple string, i.e., a weak password like “a” or “password”. Copy the hash value and search it with Google.



In the example above you can see that there are more than 29,000 webpages that have the hash value corresponding to “`a`” stored. A hacker would not take a lot of time to find out a password like that.

b) Generate several more hash values of password that could be more complicated (for example `thisismypassword`) or combinations of letters and numbers (for example `james22`) or combinations of letters, numbers, and symbols (for example `james22!`). You could try passwords that you or someone else have used in the past and search them (*if you try your password and it is stored somewhere you should change that password immediately*).

c) Continue testing different strings; add complexity like using upper and lower case, adding numbers and special characters. When do you reach a level of complexity so that the password has not been stored? Would `thisIsMyPassword` be more secure than `thisismypassword`?

Once a hash value has been deciphered (or cracked, depending on the point of view) and the plain text is known, these are stored in ***rainbow tables*** (https://en.wikipedia.org/wiki/Rainbow_table) that are normally used for cracking password hashes. Rainbow tables have gone a bit out of fashion with powerful GPUs that can crack passwords much faster than before, but still, if you can save some time by using a table instead of brute force, why wouldn’t you? For more information on tables, try: `https://project-rainbowcrack.com/` (not working at the time of writing) or `http://project-rainbowcrack.com` (working at the time of writing)

In the figure below, you can see that `thisismypassword` had already been cracked, many years ago, together with several million other common strings.

Added:	Mon 28th Feb,2011 12:21 pm	Hash:	06adcb2190010cca0f3021f0b4e6fec0	Plain:	london1234
Added:	Mon 28th Feb,2011 12:21 pm	Hash:	28f16bb7f166ab1eeab6a144ab53efd3	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:21 pm	Hash:	284d5f46f96796293a2f07b13138b220	Plain:	015415
Added:	Mon 28th Feb,2011 12:22 pm	Hash:	83062b3d6e7fe429040297c50e913b31	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:22 pm	Hash:	20a6388d8cc37eae8b65528eea1f070e	Plain:	45454545b
Added:	Mon 28th Feb,2011 12:22 pm	Hash:	83ca7d899f30f85e135c93dbaffea2d0	Plain:	jackier1p
Added:	Mon 28th Feb,2011 12:22 pm	Hash:	73f53a40475fdf651b2e82a3de50c500	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:22 pm	Hash:	e307d0ef7d5fa9cd9bddcb907fd66882	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:22 pm	Hash:	0bfb6bf696fa773183ecd4014025cd96	Plain:	3212121
Added:	Mon 28th Feb,2011 12:22 pm	Hash:	a0fcf227e7de2e055ff4b992d7adf8cc	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:23 pm	Hash:	135cbe9eefea1f11ae298bee6ea3c963	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:23 pm	Hash:	31435008693ce6976f45dedc5532e2c1	Plain:	thisismypassword
Added:	Mon 28th Feb,2011 12:23 pm	Hash:	7641bbfb58bc6dc2cf4182cb31c06469	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:23 pm	Hash:	144f0c62a8909916c759cc5e5b7bbce8	Plain:	cracking...
Added:	Mon 28th Feb,2011 12:24 pm	Hash:	acceae55b8caf53492ba722354d92af6	Plain:	indeed
Added:	Mon 28th Feb,2011 12:25 pm	Hash:	a9c73189fce66d013fb8f7d33c783f62	Plain:	cracking...

Records of the page: **67381**
Showing: **3369001 to 3369050**
Result: **5658096 occurrence(s) in 113162 page(s)**

<< Previous Next >>
< **[67381]** 67382 67383 67384 67385 67386 67387 67388 67389 67390
Previous 10 Pages Next 10 Pages

Still, there are others not yet cracked. I wonder if a password that I have used is somewhere there …

Task 3: The salt to the rescue

So, even complex passwords with numbers and letters can be cracked … what can we do? There is a simple way to avoid a hash value being decoded back to the original data with which it was generated, this is called "`salting`" or adding a second data when generating the hash value. This value, commonly known as `salt`, is unknown to the user who provides the first data. The salt can change, imagine that the system uses the time at which a data is introduced as `salt`. Or it can be the IP address from which the user has connected, or the day. Thus, when the hash value is generated, it not only depends on the original `password`, but also on the `salt`. Imagine that instead of using the password `password`, you would use `password31December2018` or `password12March2020`. Even if the salt is known, to be able to decipher the hash value would imply a significantly higher computational time to combine two sets of data, one for the password and one for the salt.

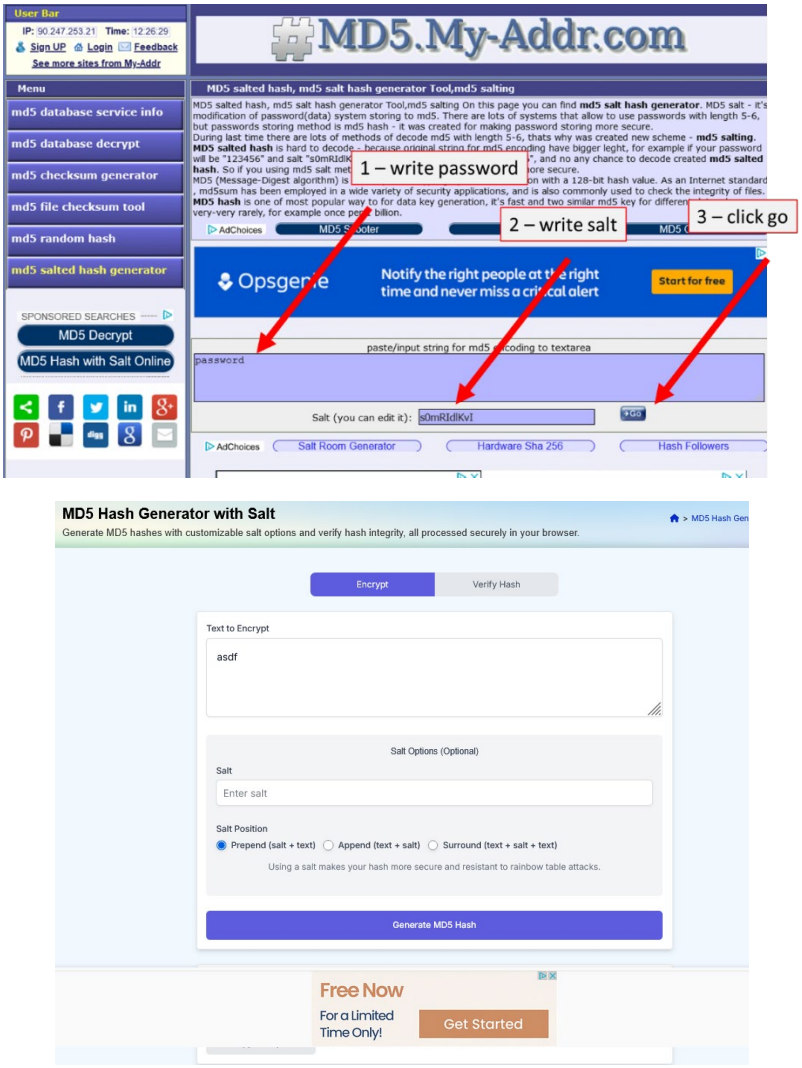
You could use some of the previous websites to generate a hash value using a `password` and `salt` values, but there are other sites where you can enter these separately, for example, try one of the following sites:

http://md5.my-addr.com/md5_salted_hash-md5_salt_hash_generator_tool.php

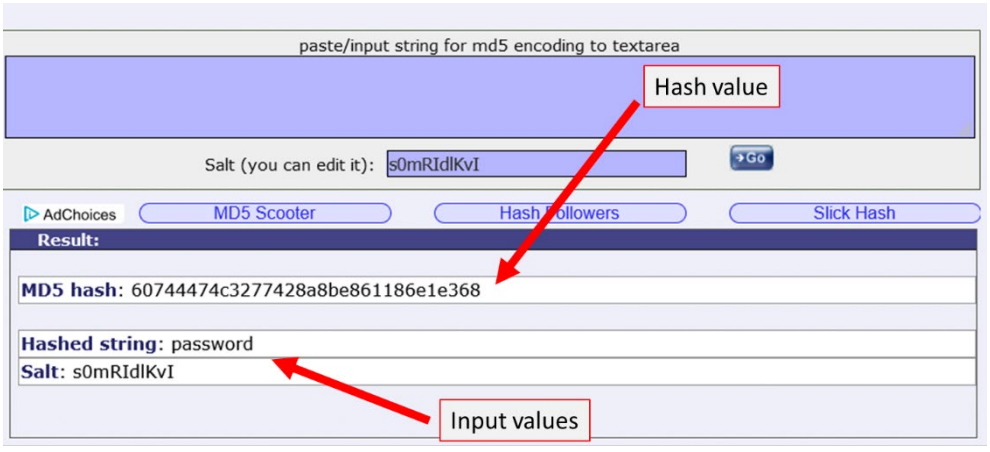
https://randommer.io/Hash/MD5

https://webutility.io/md5-hash-generator-with-salt

Like many websites, these have many distracting elements. The images below illustrate one example.



After you click go, you will get to a screen like this one:



a) Try generating hash values with some of the strings you tried previously and search for them in google. Can you find a case where the hash value is still stored somewhere?

b) Compare the results from `md5.my-addr.com` with those of `md5hashgenerator.com` do you get the same hash values with the same input data?

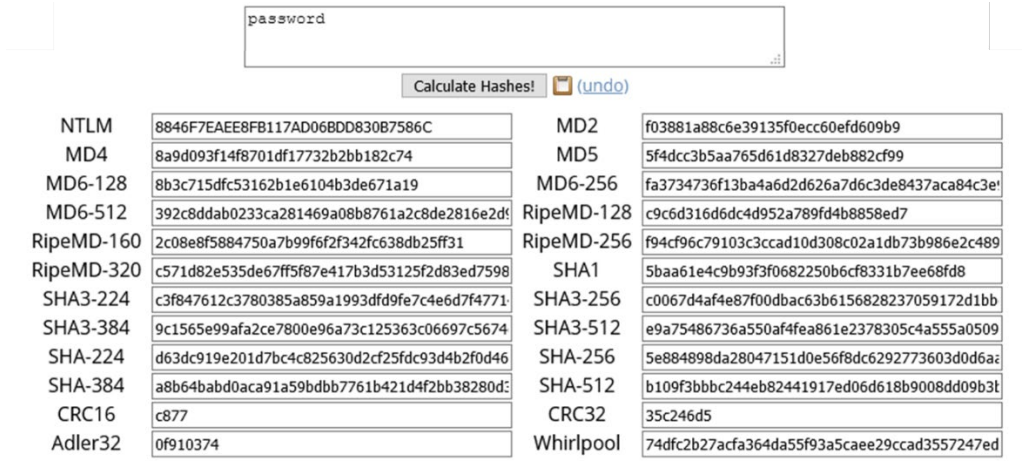
c) Try a few combinations changing the `salt` and keeping the `password`. Even with common words for salt, and a known password, it would be very difficult to crack a combination that was generated with `salt`.
d) Can you think of another benefit of using salt to generate hash values for passwords?

Task 4: Algorithms other than MD5

MD5 has been compromised and should not be used, but it is still a very good tool for illustration purposes. Are there any other hashing algorithms? Yes indeed, have a look at:

`https://www.browserling.com/tools/all-hashes`

where you can hash a string with 26 different algorithms.



a) Try generating a few hash values and observe the length of the values. Remember, the longer they are, the more difficult and more time consuming it will be to be cracked.
b) Hang on, the longer the better, so why are some codes very short like `CRC16`? Find out what are `CRC16` and `CRC32` and why they are so short as compared with others in that table.
c) Find out which is the most secure hashing algorithm, or at least more secure than MD5.